



# Transformation et manipulation des données

Dr Khadidja Henni

**Séance 3:** Transformation des  
données (gestion des  
anomalies et nettoyages)

# Plan de la séance

- Les données
- Prétraitement de données
- Laboratoire formatif

# Retour sur les généralités du machine learning

# Qu'est ce que le machine learning

Le paradigme de programmation classique a évolué, ainsi,

1. Avant, **programmer consiste à prescrire une logique** pour faire exécuter (une tâche).
2. Maintenant, on programme **pour rendre intelligent**, autrement dit, **programmer pour faire exécuter de manière intelligente des tâches nouvelles réputées nécessitant du raisonnement et un jugement**.
3. Par un tel mécanisme de programmation, nous serons en mesure de **faire doter un programme d'une aptitude d'apprentissage**.

## Exemple:

- Supposons qu'une entreprise veut connaître le montant total dépensé par un client ou une cliente à partir de ses factures.
  - Il suffit d'appliquer un algorithme classique, à savoir une simple addition, donc un algorithme d'apprentissage n'est pas nécessaire.
- Supposons maintenant que l'elle veut utiliser ces factures pour déterminer quels produits le client est le plus susceptible d'acheter dans un mois. Bien que cela soit vraisemblablement lié, nous n'avons manifestement pas toutes les informations nécessaires pour ce faire. Cependant,
  - Si nous disposons de l'historique d'achat d'un grand nombre d'individus, il devient possible d'utiliser un algorithme de machine learning pour qu'il en tire un modèle prédictif nous permettant d'apporter une réponse à notre question.

# Pourquoi utiliser le machine learning?

- Le machine learning peut servir à résoudre des problèmes
  - Que l'on ne sait pas résoudre (comme dans l'exemple de la prédiction d'achats);
  - Que l'on sait résoudre, mais dont on ne sait formaliser comment les résoudre en termes algorithmiques (c'est le cas par exemple de la reconnaissance d'images ou de la compréhension du langage naturel) ;
  - Que l'on sait résoudre, mais avec des procédures beaucoup trop gourmandes en ressources informatiques (c'est le cas par exemple de la prédiction d'interactions entre molécules de grande taille, pour lesquelles les simulations sont très lourdes).

Le machine learning est donc utilisé quand les données sont abondantes (relativement), mais les connaissances peu accessibles ou peu développées.

# Ingrédients du machine learning

Le machine learning repose sur deux piliers fondamentaux :

1. Les **données**, qui sont les exemples à partir duquel l'algorithme va apprendre;
  2. L'*algorithme d'apprentissage*, qui est la procédure que l'on fait tourner sur ces données pour produire un modèle.  
On appelle *entraînement* le fait de faire tourner un algorithme d'apprentissage sur un jeu de données.
- Ces deux piliers sont aussi importants l'un que l'autre. D'une part, aucun algorithme d'apprentissage ne pourra créer un bon modèle à partir de données qui ne sont pas pertinentes – c'est le concept **garbage in, garbage out** qui stipule qu'un algorithme d'apprentissage auquel on fournit des données de mauvaise qualité ne pourra pas faire autre que des prédictions de mauvaise qualité.
  - D'autre part, un modèle appris avec un algorithme inadapté sur des données pertinentes ne pourra pas être de bonne qualité. Ceci met la lumière sur l'importance du travail d'un data scientist, consistant à préparer les données afin d'éliminer les données aberrantes, gérer les données manquantes, choisir une représentation pertinente (features ingeenering), etc.

# Ingrédients du machine learning

- *Algorithme d'apprentissage automatique VS modèle appris*

Bien que l'usage soit souvent d'appeler les deux du même nom, il faut distinguer l'algorithme d'apprentissage automatique du modèle appris : le premier utilise les données pour produire le second, qui peut ensuite être appliqué comme un programme classique.

# La transformation de données



# Qu'est ce que la transformation de données?

## Définition

La transformation de données consiste à **modifier la structure, le format ou les valeurs** d'un jeu de données pour le rendre **exploitable par des algorithmes** d'analyse ou d'apprentissage automatique. Cela inclut des opérations telles que :

- Le **nettoyage** (suppression ou correction d'erreurs)
- La **normalisation** ou **mise à l'échelle**
- L'**encodage** de données catégorielles
- La **restructuration** des tables (pivot, fusion...)

Elle peut être **automatique** (via des pipelines) ou **manuelle** (via des scripts ou interfaces graphiques).

## Rôle dans le cycle de la data science

La transformation intervient **après la collecte** des données et **avant la modélisation**. Elle fait partie intégrante du **pipeline de traitement** des données.

### Cycle typique :

- **Collecte** (bases de données, capteurs, API, fichiers CSV...)
- **Transformation et nettoyage**
- **Exploration visuelle / descriptive**
- **Modélisation** (prédiction, classification...)
- **Évaluation et déploiement**

**Une donnée bien transformée = une analyse plus fiable**

# Lien entre le prétraitement et l'analyse exploratoire

## Lien entre le prétraitement et l'analyse exploratoire

- **Transformation  $\neq$  Analyse exploratoire (EDA)**
  - Transformation : opérationnelle (ex : changer les types, corriger les valeurs)
  - EDA : descriptive (ex : comprendre la distribution, les corrélations)
- **Prétraitement** : terme souvent utilisé en apprentissage machine pour désigner l'ensemble des opérations **avant l'entraînement du modèle**.

En pratique, transformation, nettoyage et prétraitement sont souvent **imbriqués**.

# Pourquoi c'est crucial ?

## Le principe du "Garbage In, Garbage Out »

- Si les données sont erronées ou mal préparées, même les meilleurs modèles produiront des résultats inutiles.
- Un modèle de prédiction sera biaisé si :
  - Les valeurs manquantes sont mal gérées
  - Les types de données sont incorrects
  - Les échelles sont incohérentes
  - Des valeurs aberrantes polluent les moyennes
- Une bonne transformation augmente la **fiabilité**, la **stabilité** et la **précision** des modèles

# Les données

# Les données

Jusqu'à **80 % d'un projet IA** concerne la collecte de données :

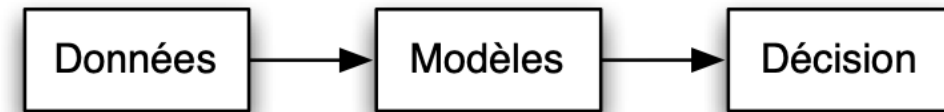
- **Quelles données sont requises ?**
- **Quelles données sont disponibles ?**
- **Comment sélectionner les données ?**
- **Comment collecter les données ?**
- **Comment nettoyer les données ?**
- **Comment préparer les données ?**
- **Comment utiliser les données ?**

Les données sont une **collection de faits**.

## Les données



Pyramide DICS



Principe de l'apprentissage machine

# Les données

- Le terme « donnée » se définit de différentes manières dans la littérature, selon les domaines et les champs d'application. Mentionnons quelques exemples : Une donnée est...
  - un enregistrement caractérisé par un ensemble de champs (terminologie des bases de données).
  - un **individu** défini par un ensemble de caractéristiques ou de variables (terminologie issue de la statistique et du forage de données).
  - une **forme** définie par un ensemble de caractéristiques ou de variables (terminologie issue de la reconnaissance de formes).
  - une **instance** caractérisée par un ensemble d'attributs (terminologie orientée objet en informatique).
  - ...



# Les données

- La détermination du type de chaque variable est une étape nécessaire avant leur analyse. Cette étape permet de décider des méthodes d'analyse appropriées.
  - **Variables qualitatives** : si ses valeurs ne sont pas mesurables. Les valeurs d'une variable qualitative sont appelées modalités. Exemple: Le sexe, la profession, l'état matrimonial.
    - Une **variable qualitative est dite ordinale** si ses modalités suivent une relation d'ordre. Par exemple, une pathologie peut être « légère », « modérée » ou « sévère ». Ces valeurs peuvent être ordonnées : légère < modérée < sévère, donc on parle d'une variable ordinale.
    - Une **variable qualitative est dite nominale** si ses modalités ne sont pas ordonnées naturellement. Par exemple, dans une population de personnes actives, la profession est une variable nominale
  - **Variables quantitatives**
    - Une **variable quantitative** est dite **discrète** si elle ne peut prendre que des valeurs qui peuvent être énumérées. Par exemple, le nombre de professeurs dans un département ou le nombre d'étudiants dans une classe.
    - La **variable quantitative** est dite **continue** si ses valeurs potentielles ne peuvent pas être énumérables (ou sont difficilement énumérables).

# Les données

- **Variables binaires** : sont des variables quantitatives discrètes qui possèdent des propriétés particulières. Nous distinguons deux types de données binaires :
  - **Données symétriques** : une variable binaire est dite symétrique si ses deux modalités ont la même importance, si celles-ci peuvent être indifféremment codées par 0 ou 1. Par exemple, la variable sexe peut être codées par 0 ou 1 pour masculin (de même que pour féminin).
  - **Données asymétriques** : une variable binaire est dite asymétrique si les deux modalités n'ont pas la même importance. Par exemple, le résultat d'un examen médical ne peut pas être codé par 0 (négatif) ou 1 (positif) en ce qui concerne son importance.

# Les données

| Individus | Ciel    | Temp. (en °C) | Humidité | Vent   | Jouer |
|-----------|---------|---------------|----------|--------|-------|
| J1        | Soleil  | 38            | Élevée   | Faible | Non   |
| J2        | Soleil  | 39            | Élevée   | Fort   | Non   |
| J3        | Couvert | 37.7          | Élevée   | Faible | Oui   |
| J4        | Pluie   | 20            | Élevée   | Faible | Oui   |
| J5        | Pluie   | 15            | Normale  | Faible | Oui   |
| J6        | Pluie   | 18            | Normale  | Fort   | Non   |
| J7        | Couvert | 18            | Normale  | Faible | Oui   |
| J8        | Soleil  | 21            | Élevée   | Faible | Non   |
| J9        | Soleil  | 15            | Normale  | Faible | Oui   |
| J10       | Pluie   | 21            | Normale  | Fort   | Oui   |
| J11       | Soleil  | 23.5          | Normale  | Fort   | Oui   |
| J12       | Couvert | 23            | Élevée   | Fort   | Oui   |
| J13       | Couvert | 40            | Normale  | Faible | Oui   |
| J14       | Pluie   | 20            | Élevée   | Fort   | Non   |

- L'analyse de ce tableau permet de voir que nous disposons de 14 individus (14 jours : J1 à J14). Chaque individu est caractérisé par 4 variables qui correspondent aux conditions météorologiques (État du ciel, Température, Humidité dans l'air, Force du vent).
- La classe Jouer peut prendre les valeurs **Oui** ou **Non**.

## Description des variables.

| Variables | Type                  | Unités/Modalités       |
|-----------|-----------------------|------------------------|
| Ciel      | Qualitative           | Soleil, Couvert, Pluie |
| Temp.     | Quantitative continue | °C                     |
| Humidité  | Qualitative           | Élevée, Normale        |
| Vent      | Qualitative           | Fort, Faible           |

# Les données

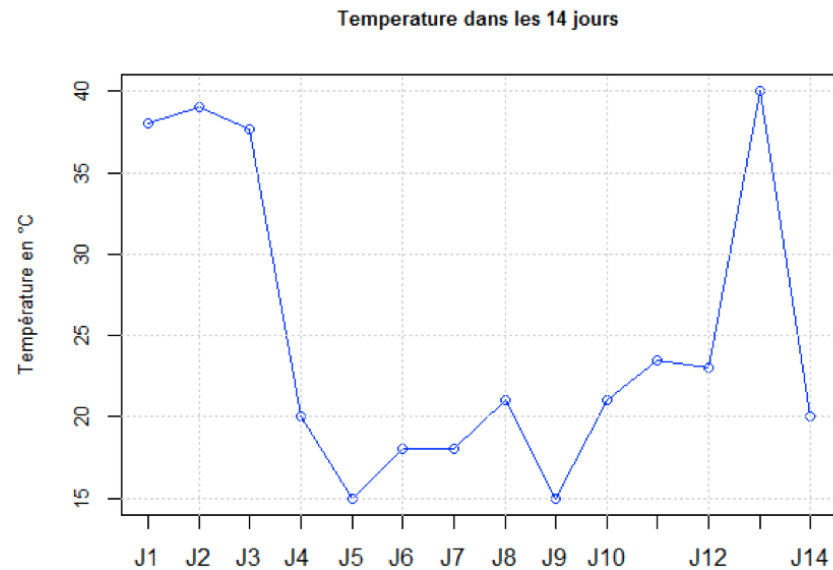
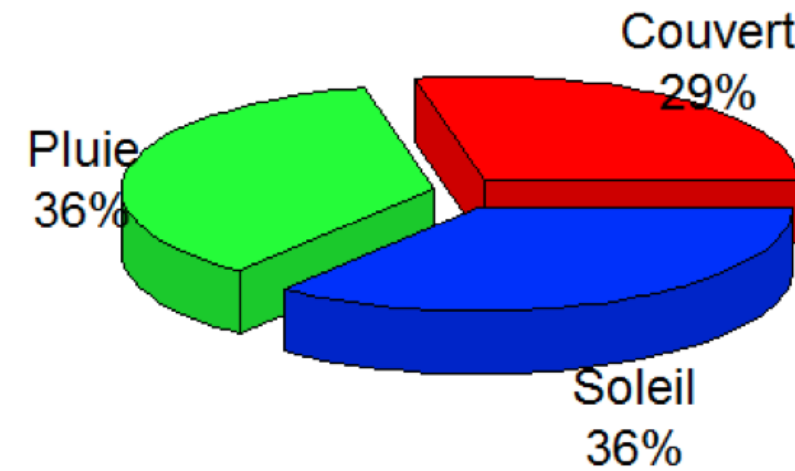


Diagramme circulaire pour la variable 'Ciel'



# Les données

- On note la taille du jeu de données  $N$ , chaque observation  $\mathbf{x}_n$  et le jeu de données de  $N$  observations  $(\mathbf{x}_n)_{1 \leq n \leq N}$ .
- Deux grandes familles de jeux de données peuvent être utilisées :
  - Les données étiquetées: chaque observation  $\mathbf{x}$  n'est fournie avec une étiquette (*label*)  $y_n$ ;
  - Les données non-étiquetées: comme le nom l'indique, aucune étiquette n'est fournie.
- En général, c'est plus facile de constituer un jeu de données non étiquetées qu'un jeu de données étiquetées. Dans le premier cas, il « suffit » de collecter des données après prétraitement automatique minimal, alors que dans le second cas une intervention humaine potentiellement couteuse est souvent nécessaire pour définir les étiquettes.
- Les données peuvent être vues comme des points dans un certain espace. Il est souvent nécessaire de comparer ces points, et il est alors bien pratique que l'espace des données soit muni d'une distance. Dans le cas de données décrites dans un espace vectoriel, les normes usuelles  $\|\cdot\|_1$  ou  $\|\cdot\|_2$  (norme euclidienne) font souvent l'affaire. Rappelons que si  $\mathbf{x}$  est un vecteur de  $\mathbb{R}^d$ , de composantes  $(x_1, \dots, x_n)$ , avec  $|x_i|$  désigne la valeur absolue de la  $i$ -ème composante. Chaque composante  $x_i$  de l'observation  $\mathbf{x}$  est aussi appelée attribut, dimension, caractéristique (feature), ou variable.

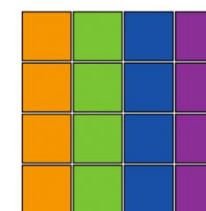
$$\|\mathbf{x}\|_1 = \sum_{i=1}^d |x^i| \quad \text{et} \quad \|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^d |x^i|^2}$$

# Les représentations des données

## • Les données structurées ?

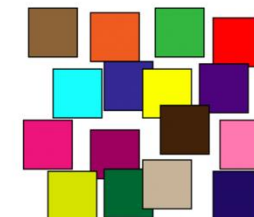
- Ce sont des informations qui ont été formatées et transformées en un modèle de données bien défini.
- Les données brutes sont mappées dans des champs prédéfinis qui peuvent ensuite être extraits et lus facilement via SQL. Les bases de données relationnelles SQL, constituées de tables avec des lignes et des colonnes, sont le parfait exemple de données structurées.
- Des exemples de données structurées incluraient des noms de clients classés par ordre alphabétique et des numéros de carte de crédit correctement organisés.

Données structurées



Informations que vous pourriez trouver dans une base de données (p. ex., noms, numéros de téléphone, dates, adresses courriel).

Données non structurées



Informations uniques et difficiles à classer (p. ex., messages texte, courriels, billets de blog, publications de médias sociaux).

## • Que sont les données non structurées ?

- Ces données non structurées sont définies comme des données présentes sous forme brute absolue.
- Ces données sont difficiles à traiter en raison de leur organisation et de leurs formatages complexes.
- La gestion des données non structurées peut prendre des données sous de nombreuses formes, y compris les publications sur les réseaux sociaux, les chats, l'imagerie satellite, les données de capteurs IoT, les e-mails et les présentations, pour les organiser de manière logique et prédéfinie dans un stockage de données.

Données externes non structurées:  
Réseaux, UGC, etc.Données externes structurées:  
Panel, Sondage, Partenaires, etc.

| person | year | income | age | sex |
|--------|------|--------|-----|-----|
| 1      | 2001 | 1300   | 27  | 1   |
| 1      | 2002 | 1600   | 28  | 1   |
| 1      | 2003 | 2000   | 29  | 1   |
| 2      | 2001 | 2000   | 38  | 2   |
| 2      | 2002 | 2300   | 39  | 2   |
| 2      | 2003 | 2400   | 40  | 2   |

Données internalisées:  
Datawarehouse, ERP, CRM, etc.Données utilisables par  
l'entreprise pour mettre en place  
des solutions

# Les données

- Le concept des données se conclut dans **les dimensions d'applications des données structurées ou non-structurées** ; dont on distingue plusieurs types avec des caractéristiques qui varient selon le domaine d'application choisi.
  - **Capacité de stockage** : La capacité des disques a augmenté par 10x alors que le prix par Gb a chuté du même ratio. Une augmentation de 100x à prix constant.
  - **Capacité d'analyse** : La loi de Moore en action pendant environ 35 ans et Plus récemment, la capacité d'analyse augmente grâce à l'ajout de cœurs dans les unités centrales
  - **Augmentation exponentielle de la quantité de données non structurées** : Email, chat, blog, web, musique, photo, vidéo, etc.
  - **Augmentation de la capacité de stockage et d'analyse** : L'utilisation de plusieurs machines en parallèle devient accessible
  - **Les technologies existantes ne sont pas conçues pour ingérer ces données** : Base de données relationnelles (tabulaires), mainframes, tableurs (Excel), etc.
  - De "nouvelles" technologies et techniques d'analyse sont nécessaires.

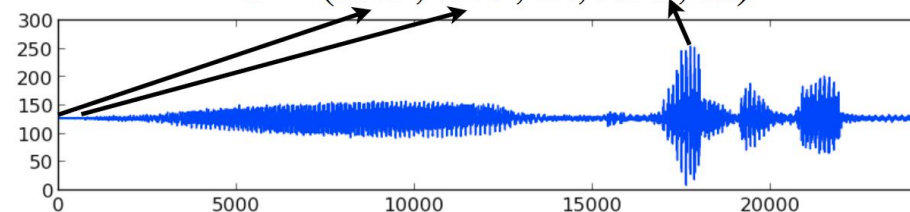
# Les données

Données d'entraînement

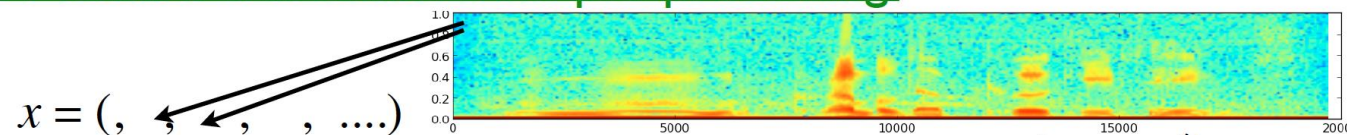
Raw representation:

$$x = (0, 0, \dots, 54, 120, \dots, 0, 0)$$

$$x = (125, 125, \dots, 250, \dots)$$



Or feature extraction from pre-processing:



$$x = (, \leftarrow, \leftarrow, , \dots)$$

Bag of words for «The cat jumped»:  $x = (\dots 0 \dots, 0, 1, \dots 0 \dots, 1, 0, 0, \dots, 0, 0, 1, 0, \dots 0 \dots)$

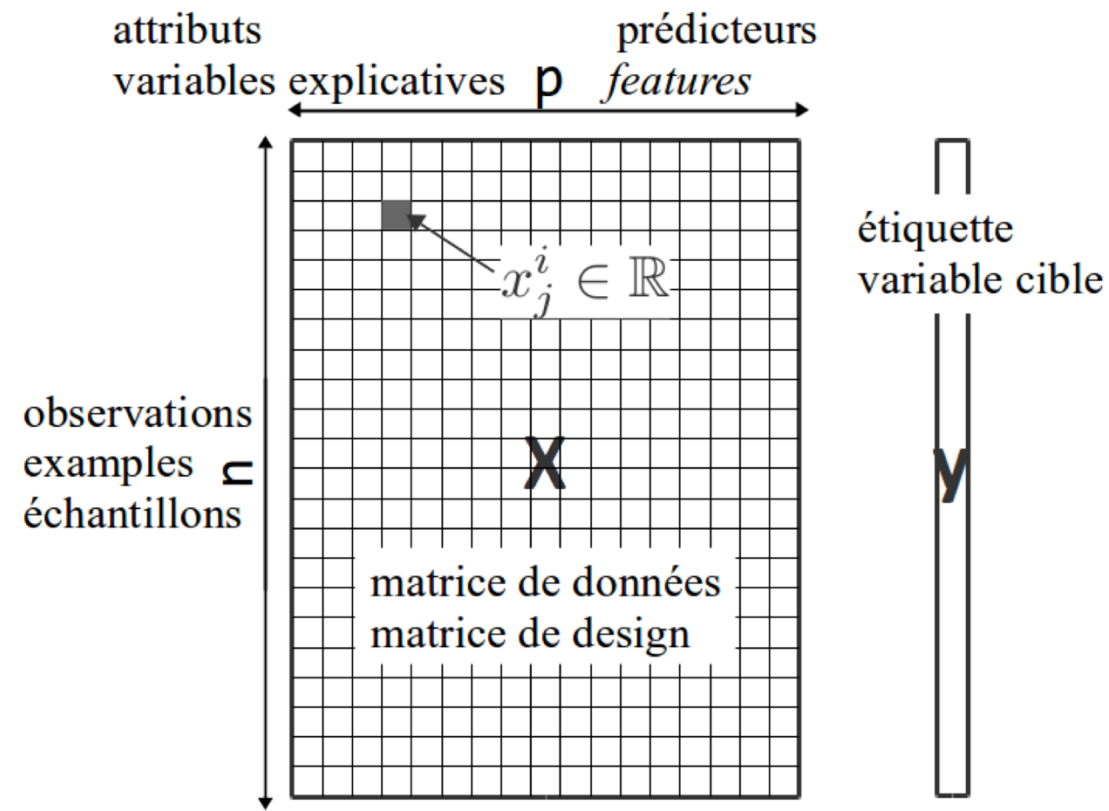
OR vector of handmade features:

ex: Histograms of Oriented Gradients

$$x = (\text{feature } 1, \dots, \text{feature } d)$$



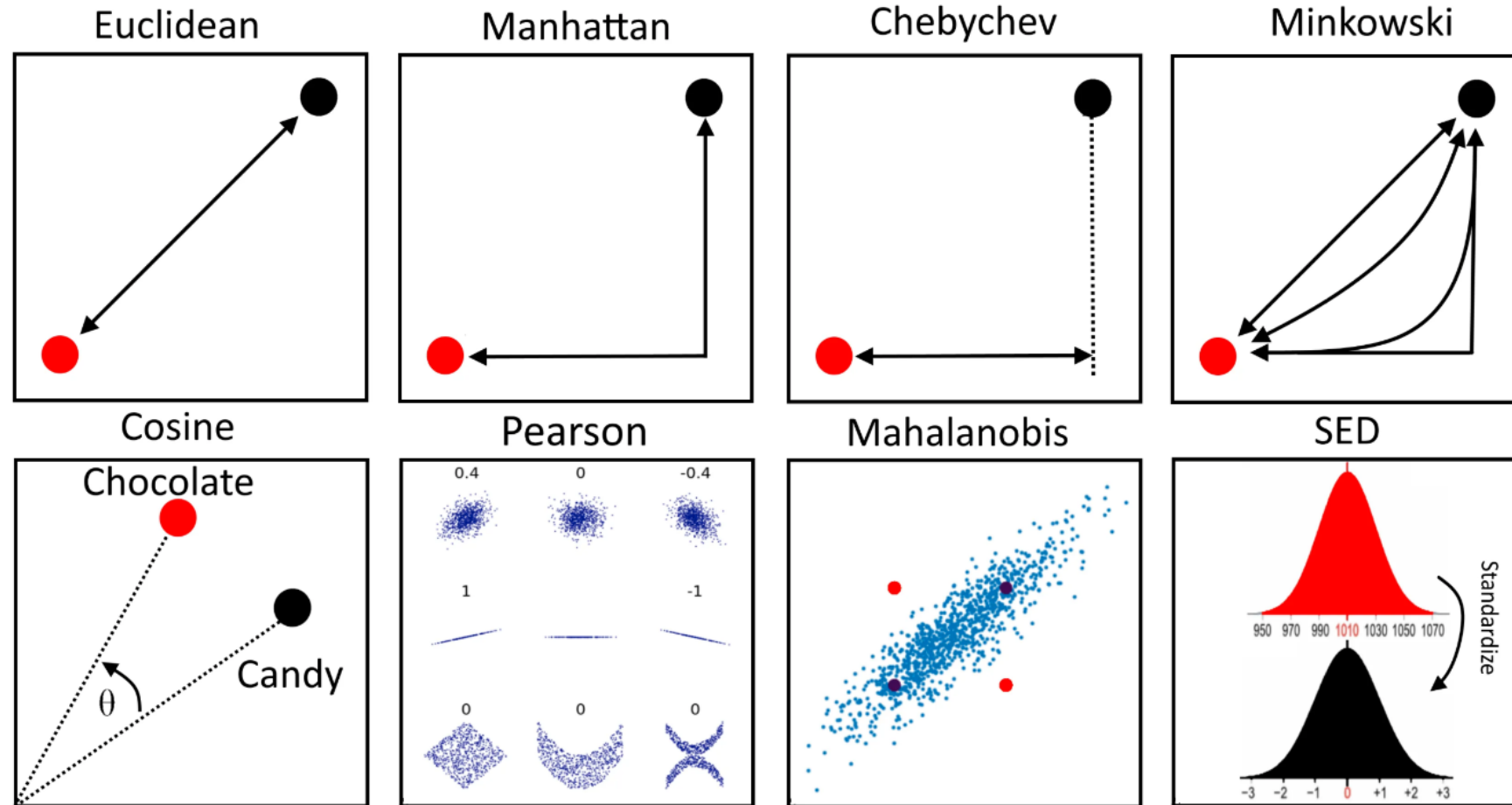
# Les données



## La similarité des données

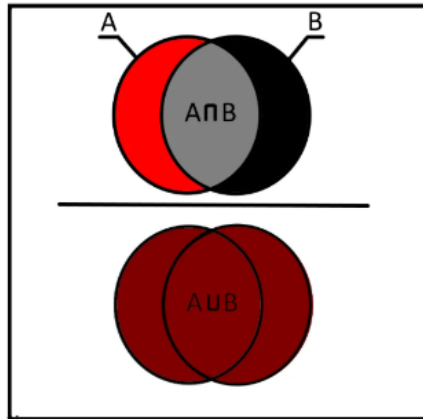
- Une distance donnée (par exemple, une dissimilarité) est considérée comme une métrique si et seulement si elle satisfait les quatre conditions suivantes :
  1. Non-négativité :  $d(p, q) \geq 0$ , pour toute paire d'observations distinctes  $p$  et  $q$ .
  2. Symétrie :  $d(p, q) = d(q, p)$  pour tous les  $p$  et  $q$ .
  3. Inégalité triangulaire :  $d(p, q) \leq d(p, r) + d(r, q)$  pour tous les  $p, q$  et  $r$ .
  4.  $d(p, q) = 0$  seulement si  $p = q$ .
- Les mesures de distance sont le principe fondamental de la classification, comme l'algorithme de classification des  $k$  plus proches voisins, qui mesure la dissimilarité entre les échantillons de données donnés. De plus, le choix d'une métrique de distance aura une influence importante sur les performances du classifieur. Par conséquent, la façon dont vous calculez les distances entre les objets jouera un rôle crucial dans les performances de l'algorithme de classification.

# La similarité des données

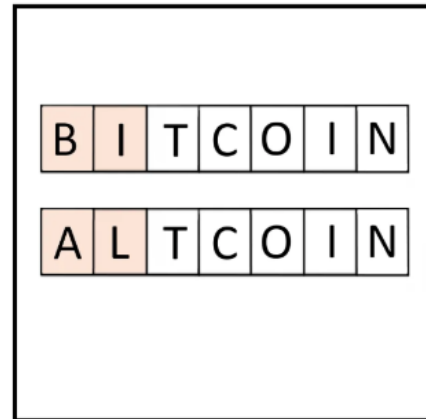


# La similarité des données

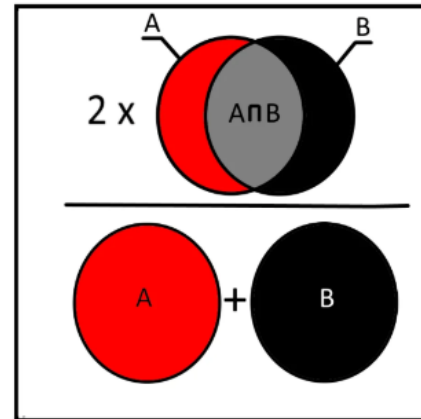
Jaccard



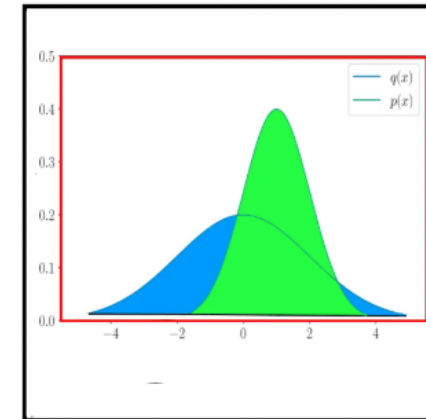
Levenshtein



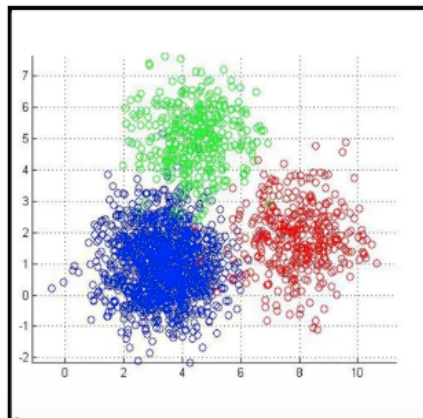
Sørensen–Dice



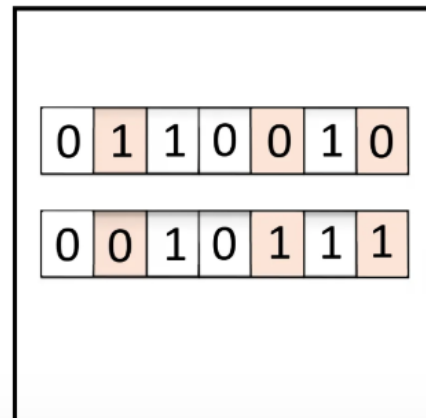
Jensen-Shannon



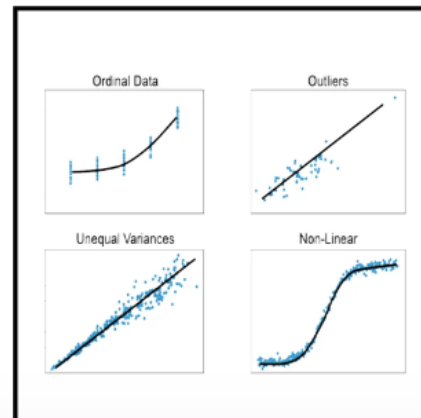
Canberra



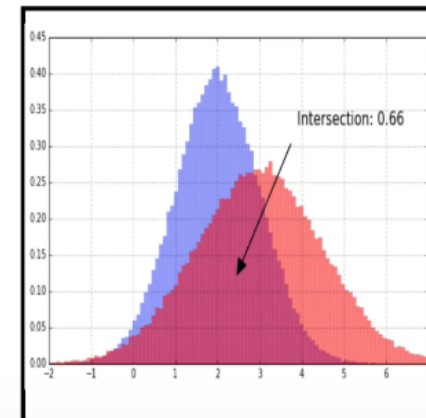
Hamming



Spearman



Chi-Square



# Les techniques de prétraitement des données

- Parmi les techniques de préparation des caractéristiques:

- 1. Gestion de données manquantes**

- 2. Gestion des valeurs aberrantes (Outliers)**

3. Binning (Discrétisation)

4. Transformation logarithmique (Log Transform)

5. Encodage one-hot (One-Hot Encoding)

6. Opérations de regroupement (Grouping Operations)

7. Séparation des caractéristiques (Feature Split)

8. Mise à l'échelle (Scaling)

9. Extraction de dates (Extracting Date)

# Traitement des données manquantes

# Nettoyage de données: données manquantes

- Les **valeurs manquantes** sont l'un **des problèmes les plus courants** que l'on **peut rencontrer lorsqu'on prépare nos données** pour l'apprentissage automatique.

## **Pourquoi les données sont-elles manquantes ?**

Les données manquantes sont très fréquentes dans les jeux de données réels. Elles peuvent résulter de :

- Problèmes techniques (erreurs de capteurs, bugs logiciels)
- Refus ou oubli de répondre à une question (ex. : âge, revenu...)
- Fusion de sources de données hétérogènes
- Collecte incomplète ou conditions d'accès variables
- Problèmes de confidentialité, etc

Il est **essentiel de comprendre la cause** avant de choisir une méthode de traitement.

Quelle que soit la raison, les valeurs manquantes affectent les performances des modèles d'apprentissage automatique. La plupart des algorithmes **n'acceptent pas** les ensembles de données contenant des **valeurs manquantes** et **génèrent une erreur**.

# Nettoyage de données: données manquantes

**Types de données manquantes:** Trois catégories (selon le mécanisme de "manquance")

## 1. Missing Completely At Random (MCAR)

**Définition :** La probabilité qu'une donnée soit manquante est **entièrement indépendante** :

- Des autres variables du jeu de données
- De la valeur elle-même (connue ou inconnue).

**Autrement dit :** l'absence est **purement aléatoire**.

**Exemple :** Un thermomètre enregistre la température chaque heure. À 3h du matin, il perd momentanément la connexion, sans lien avec la température ou un autre facteur.

Les données manquantes **ne sont pas biaisées** : elles ressemblent statistiquement à celles qui sont présentes.

**Implication :** Les données peuvent être supprimées **sans risque de biais** (ex. : `dropna()`), ou imputées simplement.



# Nettoyage de données: données manquantes

## 2. Missing At Random (MAR)

**Définition** : La probabilité qu'une donnée soit manquante **dépend d'autres variables observées** dans le dataset, **mais pas de la valeur manquante elle-même**.

**Autrement dit** : si on connaît les autres variables, on peut **expliquer** pourquoi la donnée est manquante.

**Exemple concret** : Le revenu est plus souvent manquant chez les jeunes.

La probabilité qu'une valeur de revenu soit manquante **dépend de l'âge** (variable connue), **mais pas du revenu réel**.

**Implication** : On peut utiliser des **modèles d'imputation** qui exploitent les autres variables (KNN, régression, etc.).

# Nettoyage de données: données manquantes

## 3. Missing Not At Random (MNAR)

**Définition :** La probabilité qu'une donnée soit manquante **dépend directement de la valeur elle-même**, c'est-à-dire de **ce qu'on ne voit pas**.

**Autrement dit :** la valeur est **volontairement ou structurellement masquée** parce qu'elle est extrême, sensible ou taboue.

**Exemple concret :** Les personnes ayant un **revenu très faible** sont plus susceptibles de **ne pas déclarer** leur revenu.

Donc **plus la valeur est basse, plus elle a de chances d'être manquante**.

**Implication :** Il est **très difficile de corriger cela**, car les données manquantes **ne sont pas représentatives**.

On doit parfois :

- Collecter plus d'informations
- Introduire un indicateur binaire de "donnée manquante"
- Tester plusieurs hypothèses via modélisation

# Nettoyage de données: données manquantes

## Détection des valeurs manquantes

Avant de traiter les valeurs manquantes, il faut **les détecter, les localiser et en mesurer l'impact**. Cela permet de choisir la **stratégie de traitement appropriée** (suppression, imputation, transformation...).

### Pourquoi c'est important ?

- Certaines colonnes peuvent avoir **plus de 50 % de valeurs manquantes**.
- D'autres ne contiennent que **quelques lignes vides**, et peuvent être facilement imputées.
- Un **diagnostic précis** évite les suppressions excessives ou des imputations inutiles.

# Nettoyage de données: données manquantes

## Outils de détection avec pandas (Python)

### 1. Comptage des valeurs manquantes par colonne (**isnull()** / **.sum()** )

```
df.isnull().sum()
```

- Affiche, pour chaque colonne, le **nombre de valeurs nulles** (NaN)
- Pratique pour repérer les colonnes les plus problématiques

On peut combiner avec un `sort_values()` pour les classer :

```
df.isnull().sum().sort_values(ascending=False)
```

### 2. Résumé global du dataset (**df.info()**)

```
df.info()
```

- le **nombre total de valeurs par colonne**
- le **type de données**
- si certaines colonnes ont **des entrées manquantes**

Permet de voir rapidement si une colonne object est mal typée (ex. : dates non reconnues).

# Nettoyage de données: données manquantes

## 3. Pourcentage de valeurs manquantes

- Pour mesurer l'impact réel (et pas seulement le nombre brut) :
- Calcule la **proportion (%) de valeurs manquantes** par colonne

```
(df.isnull().mean() * 100).sort_values(ascending=False)
```

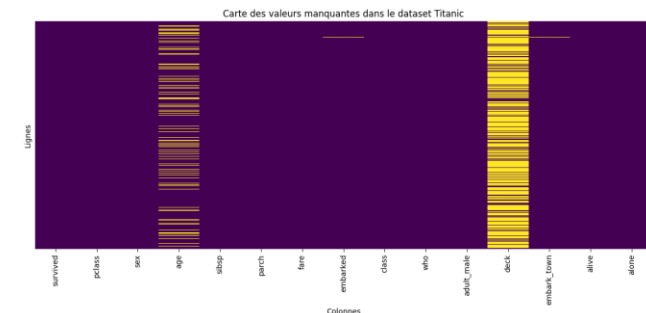
## 4. Visualisation avec seaborn : heatmap

Cela produit une **carte visuelle** où :

- Chaque **ligne** correspond à un enregistrement
- Les **valeurs manquantes** apparaissent en couleur
- Très utile pour repérer :
  - Des motifs de manquant
  - Des dépendances entre colonnes

```
import seaborn as sns
import matplotlib.pyplot as plt

sns.heatmap(df.isnull(), cbar=False, cmap="viridis")
plt.title("Carte des valeurs manquantes")
plt.show()
```



# Nettoyage de données: données manquantes

## Suppression

**Description :** Supprimer les lignes ou les colonnes contenant des valeurs manquantes.

**Quand l'utiliser :**

- **Lignes** : si  $< 5 \%$  de lignes concernées et données **MCAR**
- **Colonnes** : si  $> 70 \%$  de valeurs manquantes

Éviter de supprimer pour données MAR/MNAR, car ca consiste aussi a supprimer une relation implicite entre les variables, ce qui biaisera l'échantillon.

```
# Supprimer lignes avec NaN
df_clean = df.dropna()

# Supprimer colonnes trop vides (>70%)
seuil = 0.7
df_clean = df.dropna(axis=1, thresh=int((1 - seuil) * len(df)))
```

# Nettoyage de données: données manquantes

## Suppression

- **Exemple MAR** : Les valeurs sont manquantes en fonction d'autres variables connues, mais pas de la valeur elle-même. Si on supprime ces lignes, on **supprime aussi une relation implicite entre les variables**, ce qui **biaisera l'échantillon**.
- Un hôpital collecte des données sur les patients : âge, sexe, revenu.
- Le **revenu est souvent manquant chez les patients jeunes**, mais pas chez les plus âgés.

| Âge | Sexe | Revenu |
|-----|------|--------|
| 22  | M    | NaN    |
| 24  | F    | NaN    |
| 35  | F    | 55 000 |
| 60  | M    | 40 000 |

**Si on supprime toutes les lignes avec NaN dans revenu, on éliminera tous les jeunes patients du dataset.**

**-> Le modèle ne saura rien des jeunes → biais de sélection.**

# Nettoyage de données: données manquantes

## Suppression

- **Exemple MNAR** : La donnée est manquante à cause de sa propre valeur, c'est-à-dire **plus la valeur est extrême, plus elle a de chances d'être absente**. Si on supprime ces lignes, on supprimera **systématiquement les cas extrêmes**, ce qui fausse complètement la distribution
- Même dataset : âge, sexe, revenu. Les personnes avec **revenu très faible refusent de le déclarer**.

| Âge | Sexe | Revenu |
|-----|------|--------|
| 28  | M    | NaN    |
| 42  | F    | 15 000 |
| 53  | M    | 60 000 |
| 37  | F    | NaN    |

Supprimer les lignes avec NaN = supprimer **les revenus les plus bas** (ceux que les gens cachent).

-> Le modèle croira que tout le monde gagne bien sa vie, donc il fera de mauvaises prédictions.



# Nettoyage de données: données manquantes

## Imputation

L'imputation des données manquantes consiste à **remplacer** les valeurs absentes dans un ensemble de données par **des valeurs estimées**. Idéalement, ces remplacements **ne doivent pas modifier** de manière **significative la distribution statistique** ni la **structure globale des données**.

1. **Imputation par une valeur statistique:** Dans le cas de données quantitatives, l'imputation de données manquantes peut se faire par : Une constante, La moyenne de la variable, La médiane de la variable, Le mode, valeur extrême.

Dans le cas de données qualitatives, l'imputation de données manquantes peut se faire par le mode:  
`sklearn.impute.SimpleImputer`

2. **Imputation avancé (par modèle):** Knn, regression, arbre de decision,..
3. **Imputation itérative**
4. **Indicateur binaire de manquance**

# Nettoyage de données: données manquantes

## 1. Imputation par une valeur statistique (moyenne, médiane ou mode)

**Description** : Cette méthode consiste à remplacer les valeurs manquantes (NaN) dans une variable par une **valeur statistique représentative** de l'ensemble des observations disponibles :

- **Moyenne** : adaptée aux variables numériques symétriques sans valeurs extrêmes,
- **Médiane** : préférée lorsque la distribution est asymétrique ou contient des valeurs aberrantes,
- **Mode** : utilisé principalement pour les variables catégorielles.

### Quand l'utiliser :

- Lorsque les données sont **MCAR (manquantes complètement aléatoirement)** ou **presque MAR (manquantes aléatoirement selon d'autres variables observables)** ;
- Lorsque le **taux de valeurs manquantes est modéré**, généralement compris entre **5 % et 30 %** ;
- Lorsque la variable :
  - est **peu asymétrique** (**skewness** = -0,5 à +0,5) : utilisation de la moyenne, sinon médiane
  - ou **contient des outliers** : utilisation de la médiane,
  - ou est **catégorielle** : utilisation du mode.

# Nettoyage de données: données manquantes

```
# Numérique : moyenne
df['age'] = df['age'].fillna(df['age'].mean())

# Numérique avec outliers : médiane
df['age'] = df['age'].fillna(df['age'].median())

# Catégoriel : mode
df['embarked'] = df['embarked'].fillna(df['embarked'].mode()[0])
```

## Avantages :

- Méthode simple à mettre en œuvre ;
- Préserve la taille du jeu de données.

## Limites :

- Peut **réduire la variance** de la variable ;
- Peut **introduire un biais** si les données sont MAR ou MNAR ;
- Ne tient pas compte des corrélations entre variables.

# Nettoyage de données: données manquantes

## 1. Imputation par une valeur statistique (moyenne, médiane ou mode)

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Données avec None (qu'on remplace par np.nan)
data = {
    'Age': [25, 30, np.nan, 40, 22],
    'Revenu': [50000, np.nan, 52000, 58000, np.nan],
    'Genre': ['Homme', 'Femme', 'Homme', np.nan, 'Femme']
}

df = pd.DataFrame(data)
print("DataFrame avant imputation :")
print(df)

# Imputation moyenne pour Age
imp_mean = SimpleImputer(strategy='mean')
df[['Age']] = imp_mean.fit_transform(df[['Age']])

# Imputation médiane pour Revenu
imp_median = SimpleImputer(strategy='median')
df[['Revenu']] = imp_median.fit_transform(df[['Revenu']])

# Imputation mode (valeur la plus fréquente) pour Genre
imp_mode = SimpleImputer(strategy='most_frequent')
df[['Genre']] = imp_mode.fit_transform(df[['Genre']])

# Résultat
print("DataFrame après imputation :")
print(df)
```

DataFrame avant imputation :

|   | Age  | Revenu  | Genre |
|---|------|---------|-------|
| 0 | 25.0 | 50000.0 | Homme |
| 1 | 30.0 | NaN     | Femme |
| 2 | NaN  | 52000.0 | Homme |
| 3 | 40.0 | 58000.0 | NaN   |
| 4 | 22.0 | NaN     | Femme |

DataFrame après imputation :

|   | Age   | Revenu  | Genre |
|---|-------|---------|-------|
| 0 | 25.00 | 50000.0 | Homme |
| 1 | 30.00 | 52000.0 | Femme |
| 2 | 29.25 | 52000.0 | Homme |
| 3 | 40.00 | 58000.0 | Femme |
| 4 | 22.00 | 52000.0 | Femme |

# Nettoyage de données: données manquantes

## 1. Imputation par une valeur statistique (moyenne, médiane ou mode)

### b. Imputation catégorique :

Remplacer les valeurs manquantes par la **valeur la plus fréquente dans une colonne** est une bonne option pour traiter les **colonnes catégorielles**. Cependant, si vous estimez que **les valeurs dans la colonne sont réparties uniformément et qu'il n'y a pas de valeur dominante**, imputer une catégorie comme "Autre" peut être plus judicieux. Dans un tel cas, votre imputation est susceptible de converger vers une sélection aléatoire, ce qui est plus adapté à la nature de la distribution des données dans la colonne.

```
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

# Exemple de données avec une valeur manquante
df = pd.DataFrame({'sexe': ['H', 'F', np.nan, 'F', 'H']})

# Imputation par la valeur la plus fréquente (ici : 'F')
imputer = SimpleImputer(strategy='most_frequent')
df_imputed = imputer.fit_transform(df)

# Conversion du résultat en DataFrame
df_imputed = pd.DataFrame(df_imputed, columns=['sexe'])

# Optionnel : forcer le type string si nécessaire
df_imputed['sexe'] = df_imputed['sexe'].astype(str)

# Affichage
print(df_imputed)
```

|   | sexe |
|---|------|
| 0 | H    |
| 1 | F    |
| 2 | F    |
| 3 | F    |
| 4 | H    |

# Nettoyage de données: données manquantes

## 2. Imputation avancé (par modèle): Knn, regression, arbre de decision,..

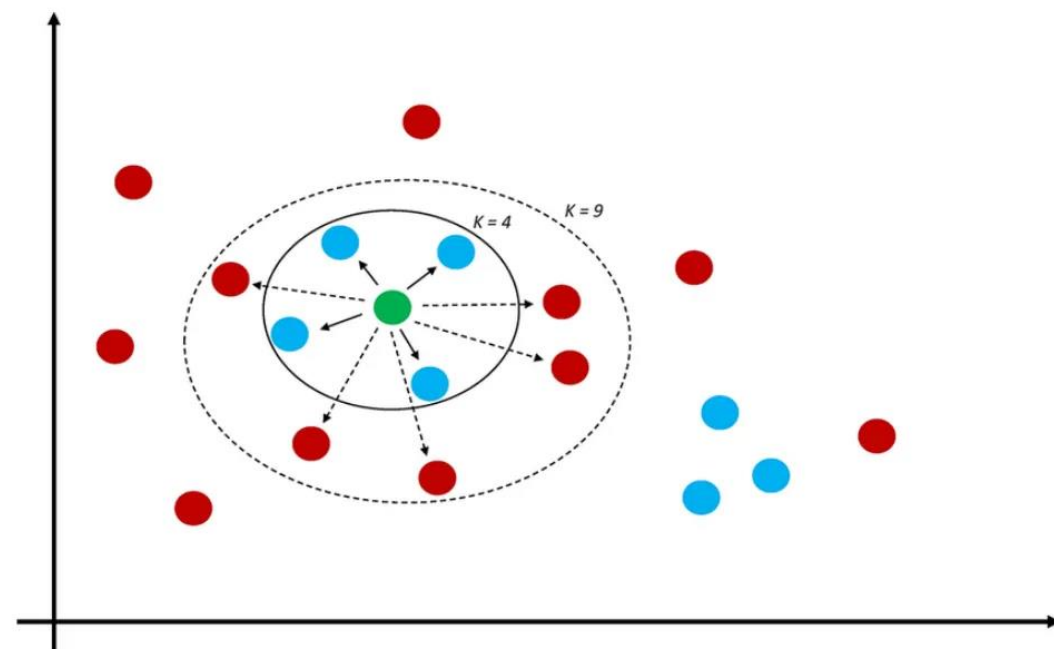
**2.1. Imputation par KNN:** L'imputation par KNN (k-plus proches voisins) consiste à **remplacer une valeur manquante par la moyenne (ou médiane)** des k observations les plus similaires selon les autres variables.

### Principe

- Pour chaque valeur manquante, on **cherche les k voisins les plus proches** en termes de distance (ex. : euclidienne).
- On calcule la **moyenne** des valeurs connues de ces voisins pour la variable à imputer.
- On remplace la valeur manquante par cette moyenne.

### Conditions

- Fonctionne pour données **numériques**.
- Peut être utilisé pour variables **catégorielles** avec codage préalable.



<https://medium.com/@hhuseyincosgun/dealing-with-missing-data-from-zero-to-advanced-4fb734ee5998>

**Remarque:** Nécessite la **normalisation** des variables (scaling), car la distance est sensible aux échelles.

# Nettoyage de données: données manquantes

## 2. Imputation avancé (par modèle): Knn, regression, arbre de decision,..

- **2.1. Imputation par régression lineaire:** On construit un **modèle de régression** pour chaque variable avec des valeurs manquantes, à partir des autres variables connues.

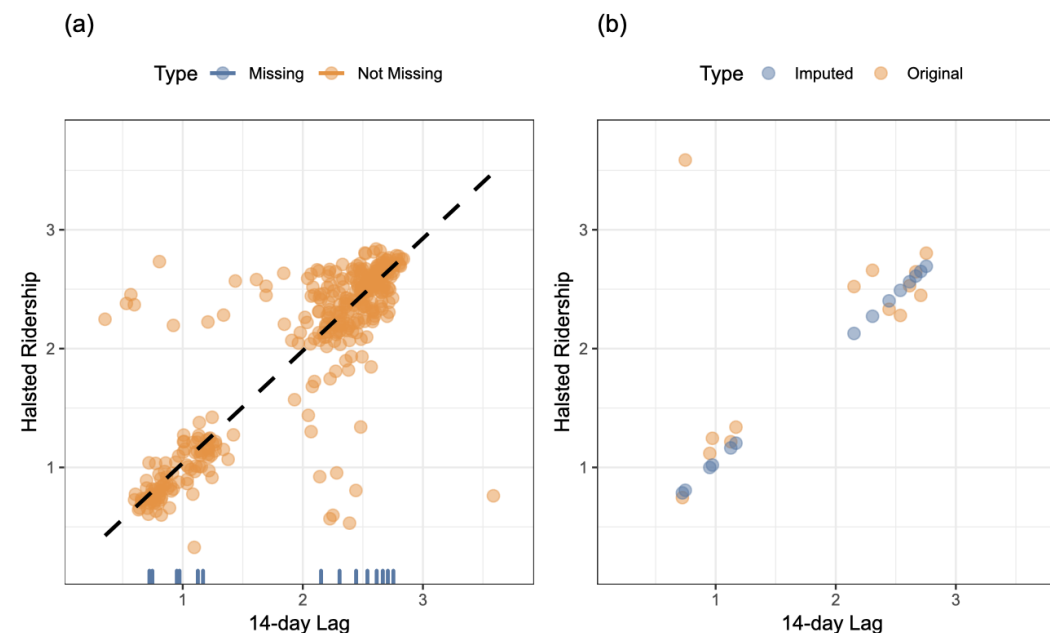
### Principe

- La variable cible est celle avec des valeurs manquantes.
- Les autres variables connues servent de **prédicteurs**.
- On entraîne le modèle sur les lignes **sans valeurs manquantes** pour cette variable.
- On **prédit** les valeurs manquantes pour les lignes incomplètes.

**Exemple** : Imputer Revenu en le modélisant comme fonction de Âge, Éducation, etc.

**Avantage** : tient compte de **relations linéaires** explicites.

**Limite** : **biais** si la relation n'est pas linéaire ou multicolinéarité.



# Nettoyage de données: données manquantes

## 2. Imputation avancé (par modèle): Knn, regression, arbre de decision,..

**2.1. Imputation par arbres de décision / random forest:** Même idée que la régression, mais ici le modèle utilisé est un **modèle d'ensemble** ou un **arbre de décision** (Random Forest, Extra Trees, etc.).

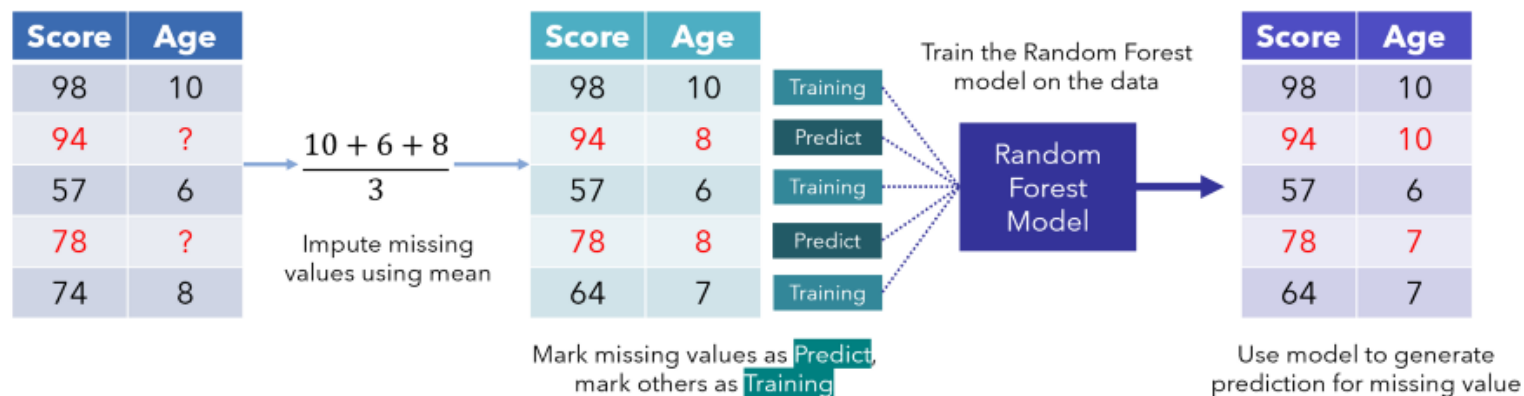
### Principe

- Plus flexible : capture **interactions non linéaires**.
- Plus robuste aux outliers.
- Peut gérer **variables numériques et catégorielles**.

### Processus

- On entraîne un modèle d'arbre pour chaque variable cible manquante.
- Le modèle apprend à prédire cette variable à partir des autres variables complètes.

**Limite :** plus **coûteux en temps de calcul**.



<https://www.analyticsvidhya.com/blog/2022/05/handling-missing-values-with-random-forest/>



# Nettoyage de données: données manquantes

## 2. Imputation avancé (par modèle): Knn, regression, arbre de decision,..

| Méthode            | Avantages                                                      | Inconvénients                                    |
|--------------------|----------------------------------------------------------------|--------------------------------------------------|
| <b>KNN Imputer</b> | Simple à comprendre, tient compte de la similarité             | Sensible aux échelles, coûteux pour grands jeux  |
| <b>Régression</b>  | Rapide, efficace si relation linéaire                          | Ne capte pas les non-linéarités                  |
| <b>Arbres / RF</b> | Gère non-linéarité, mixe variables catégorielles et numériques | Long à entraîner, nécessite tuning               |
| <b>Général</b>     | Préserve les relations entre variables, moins de biais         | Plus complexe à implémenter qu'imputation simple |

# Nettoyage de données: données manquantes

## 2. Imputation avancé (par modèle):

```
import pandas as pd
import numpy as np
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.linear_model import LinearRegression

# Jeu de données d'exemple
df = pd.DataFrame({
    'Age': [25, 30, np.nan, 40, 22],
    'Revenu': [50000, np.nan, 52000, 58000, 54000]
})

# ===== 1. IMPUTATION PAR KNN =====
knn_imputer = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(knn_imputer.fit_transform(df), columns=df.columns)

print("👉 Résultat après imputation KNN :")
print(df_knn)
```

```
# ===== 1. IMPUTATION PAR KNN =====
knn_imputer = KNNImputer(n_neighbors=2)
df_knn = pd.DataFrame(knn_imputer.fit_transform(df), columns=df.columns)

print("👉 Résultat après imputation KNN :")
print(df_knn)

# ===== 2. IMPUTATION PAR RÉGRESSION DE Age EN FONCTION DE Revenu =====

# Étape 1 : Imputation préalable de Revenu (par médiane par exemple)
revenu_imputer = SimpleImputer(strategy='median')
df['Revenu'] = revenu_imputer.fit_transform(df[['Revenu']])

# Étape 2 : Séparation des données complètes et incomplètes
df_train = df[df['Age'].notna()]
df_missing = df[df['Age'].isna()]

# Étape 3 : Régression
X_train = df_train[['Revenu']]
y_train = df_train['Age']
reg = LinearRegression()
reg.fit(X_train, y_train)

# Étape 4 : Prédiction
df.loc[df['Age'].isna(), 'Age'] = reg.predict(df_missing[['Revenu']])

print("\n👉 Résultat après imputation par régression :")
print(df)
```

# Nettoyage de données: données manquantes

## 3. Imputation itérative

- L'imputation itérative est une méthode avancée qui repose sur le **modèle de régression multivariée**. Elle consiste à **modéliser chaque variable avec des valeurs manquantes comme une fonction des autres variables**. L'imputation est ensuite effectuée **de façon séquentielle et répétée**, en réutilisant les valeurs imputées aux étapes précédentes.

### Principe

**Initialisation** : les valeurs manquantes sont d'abord imputées avec une méthode simple (moyenne, médiane...).

### **Boucle itérative :**

- On choisit une variable avec des valeurs manquantes (ex. Age).
- On entraîne un modèle (régression linéaire ou arbre, etc.) pour prédire cette variable à partir des autres.
- On met à jour les valeurs manquantes.
- On passe à la variable suivante.

Ce processus est **répété plusieurs fois** (par défaut 10 itérations) pour affiner les imputations

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# Create an instance of IterativeImputer
imputer = IterativeImputer()

# Fit and transform the DataFrame
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)

print(df_imputed)
```

# Nettoyage de données: données manquantes

## 4. Imputation avec indicateur binaire de valeurs manquantes

On **ajoute une nouvelle variable booléenne** (0 ou 1) pour chaque variable contenant des valeurs manquantes. Cette variable indique si la donnée originale était manquante ou non. Elle est ensuite utilisée **en plus** de l'imputation classique.

### Quand l'utiliser ?

- Lorsque les données sont **MAR (manquantes en fonction d'autres variables)** ou **MNAR (manquantes non aléatoirement)**.
- Quand on **souhaite préserver l'information sur l'absence elle-même**.
- Très utile dans les modèles prédictifs : l'algorithme peut apprendre que **l'absence** d'une valeur est elle-même **informative**.

```
import pandas as pd
import numpy as np

# Exemple de jeu de données
df = pd.DataFrame({'age': [25, np.nan, 30, 22, np.nan]})

# Étape 1 : Créer une colonne binaire indiquant les valeurs manquantes
df['age_missing'] = df['age'].isnull().astype(int)

# Étape 2 : Imputer la variable 'age' (par exemple par la médiane)
df['age'] = df['age'].fillna(df['age'].median())

print(df)
```

|   | age  | age_missing |
|---|------|-------------|
| 0 | 25.0 | 0           |
| 1 | 25.0 | 1           |
| 2 | 30.0 | 0           |
| 3 | 22.0 | 0           |
| 4 | 25.0 | 1           |

# Traitement des données aberrantes

## Les valeurs aberrantes (Outliers)

- Les données aberrantes (outliers) font référence à des **points atypiques** ou **inhabituels** dans un ensemble de données **qui se situent considérablement loin de la plupart des autres observations** et **peuvent influencer négativement les analyses statistiques** et les modèles d'apprentissage automatique. La **détection et la gestion des données aberrantes** sont **essentiels pour garantir l'intégrité et la fiabilité des analyses et des prédictions** basées sur les données.
- **La meilleure façon de détecter les valeurs aberrantes** est de **représenter les données visuellement**. Toutes les autres méthodologies statistiques sont susceptibles de commettre des erreurs, tandis que la **visualisation des valeurs aberrantes offre la possibilité de prendre une décision avec une grande précision**.
- **Les méthodologies statistiques sont moins précises**, mais **elles sont rapides**. Deux façons différentes de gestion des valeurs aberrantes. Ces méthodes **les détecteront en utilisant l'écart type et les percentiles**.

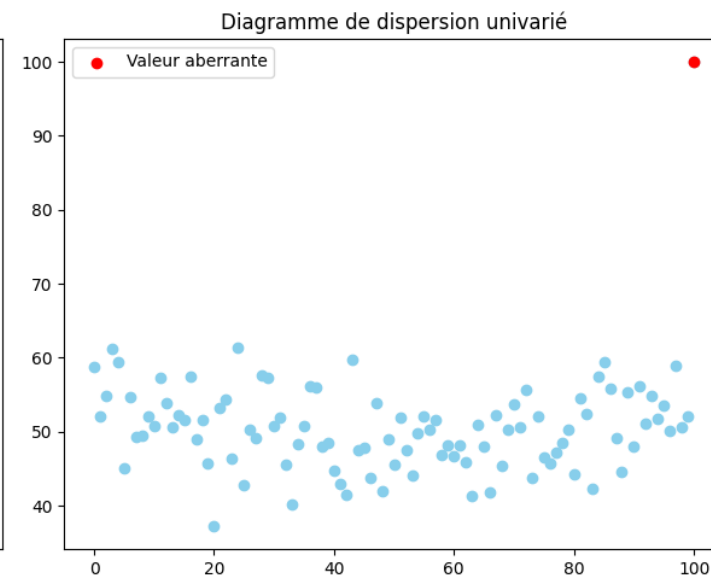
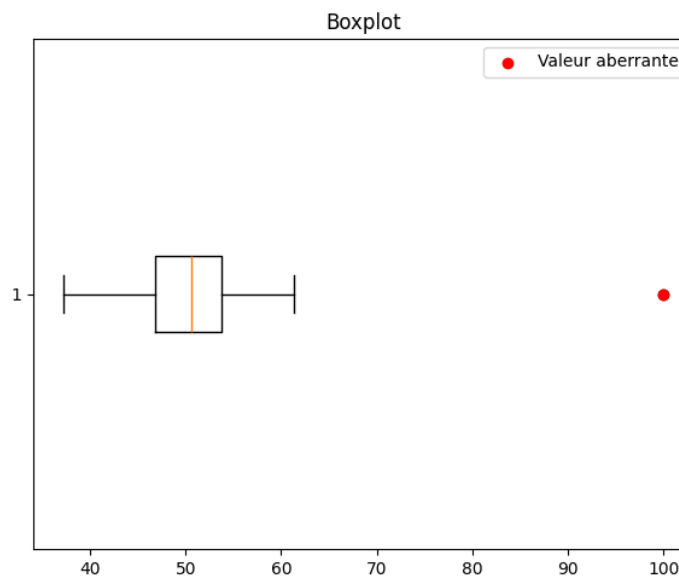
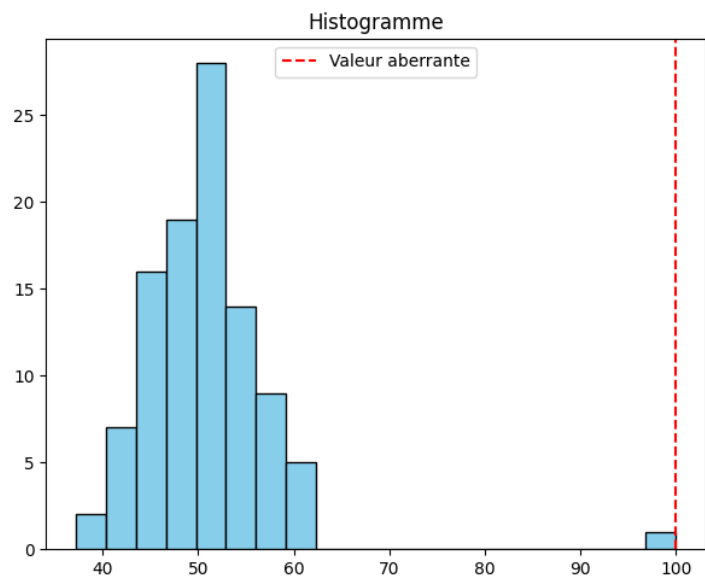
# Détection des valeurs aberrantes (Outliers)

## 1. La détection des valeurs aberrantes avec graphiques:

### 1.1. Analyse univariée

- Lorsque l'on étudie une seule variable (scalaire), certaines **méthodes graphiques** permettent d'identifier visuellement les **valeurs aberrantes**.

Parmi ces outils : **l'histogramme**, **le boxplot** (boîte à moustaches) ou encore **le diagramme de dispersion univarié** (représentation des individus en ligne).



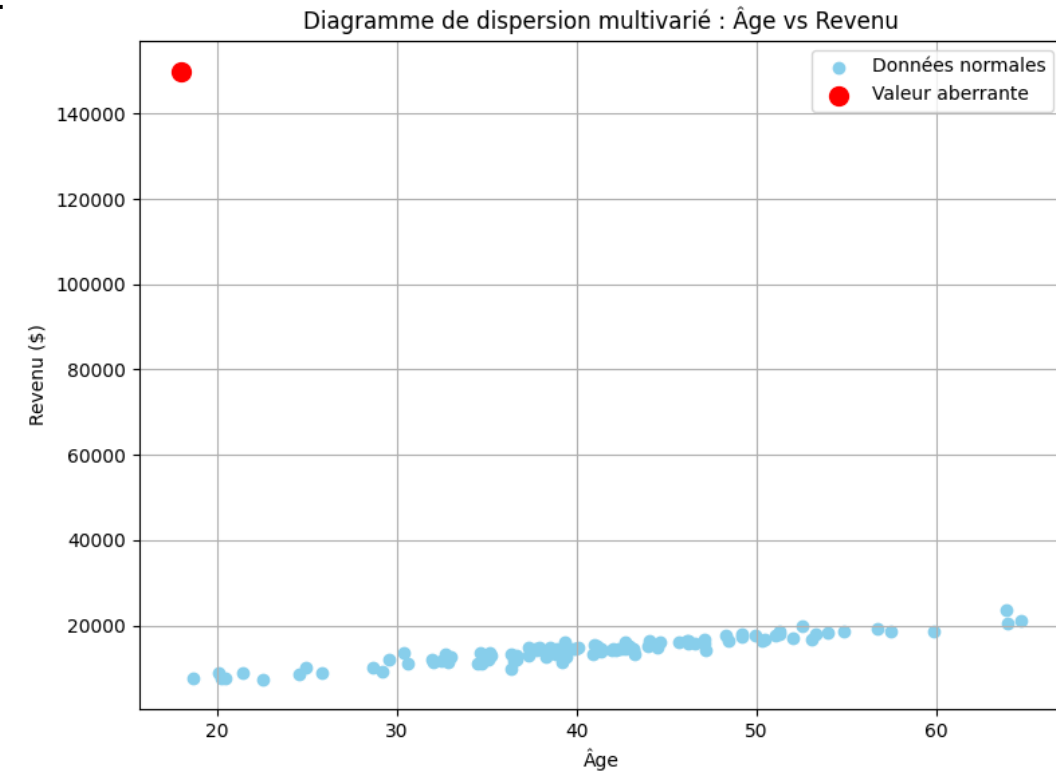
# Détection des valeurs aberrantes (Outliers)

## 1. La détection des valeurs aberrantes avec graphiques:

### 1.2. Analyse bivariée

- Le **diagramme de dispersion** (ou **scatterplot**) est utilisé pour représenter la relation entre **deux variables numériques**.
- Chaque point du nuage représente un individu.

Les valeurs aberrantes apparaissent comme des points isolés du reste du nuage.





# Détection des valeurs aberrantes (Outliers)

## 2. La détection des valeurs aberrantes avec écart type:

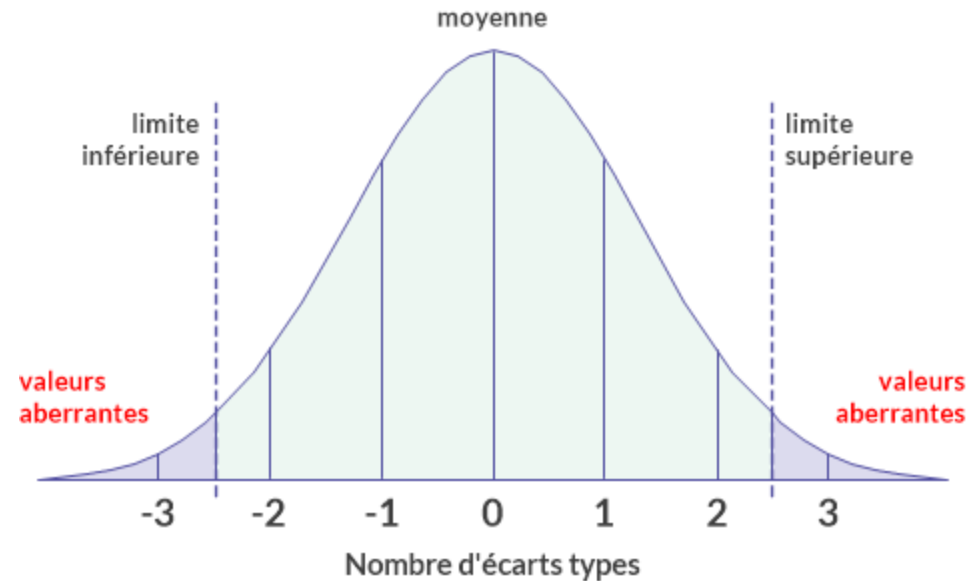
Si une observation s'écarte fortement de la moyenne, elle

peut être considérée comme aberrante.

$$\text{z-score} = \frac{x - \mu}{\sigma}$$

Si  $|z| > 2$  ou  $|z| > 3 \rightarrow$  **valeur aberrante potentielle**

- Cette méthode suppose une **distribution normale** des données.
- Elle fonctionne bien pour des **données symétriques sans trop d'outliers**.
- Pour des distributions très asymétriques, on préfère des méthodes **robustes** (IQR, MAD, Isolation Forest...).



```
import numpy as np

# Exemple simple
data = np.array([25, 26, 30, 28, 29, 100]) # 100 est suspect

# Calcul des z-scores
mean = np.mean(data)
std = np.std(data)
z_scores = (data - mean) / std

# Détection des outliers
outliers = data[np.abs(z_scores) > 2]

print("Valeurs aberrantes détectées :", outliers)
```

Valeurs aberrantes détectées : [100]

# Gestion des valeurs aberrantes (Outliers)

## **3. Le dilemme : Supprimer ou Limiter :**

Cette approche consiste à:

- a) Supprimer: repérer les valeurs aberrantes dans l'ensemble de données et les exclure complètement de l'analyse. En d'autres termes, ces points de données ne sont pas pris en compte dans les calculs statistiques ou les modèles.
- b) Limiter: identifier les valeurs aberrantes, ensuite, au lieu de les supprimer complètement, on les limite en les remplaçant par des valeurs plus proches de la moyenne ou en utilisant des seuils prédéfinis.

Le choix entre la suppression et la limitation des valeurs aberrantes dépend des objectifs de l'analyse, du type de données et de la méthode statistique utilisée.

# Gestion des valeurs aberrantes (Outliers)

## 3.1. Limiter par Capping (Winsorization)

Le **capping**, aussi appelé **Winsorization**, consiste à **remplacer les valeurs trop extrêmes par des seuils définis** (généralement les percentiles).

Autrement dit, au lieu de supprimer ou d'ignorer les outliers, on les **réduit à une valeur maximale/minimale acceptable**.

### Pourquoi?

- Pour **limiter l'influence** des valeurs aberrantes dans les statistiques (moyenne, régression...)
- Pour **préserver la taille** du dataset (contrairement à la suppression)
- Pour **stabiliser** les modèles sensibles aux extrêmes (ex. : régression linéaire)

### Quand l'utiliser ?

- Quand les outliers sont **extrêmes mais plausibles** (ex. : revenu, vente, prix)
- Quand tu veux **préserver les données mais atténuer leur impact**
- Sur des données **asymétriques** avec longue queue

# Gestion des valeurs aberrantes (Outliers)

## 3.1. Limiter par Capping (Winsorization)

| Méthode                 | Seuils courants                                           |
|-------------------------|-----------------------------------------------------------|
| Capping absolu          | Valeurs limites fixes (ex. max = 10000)                   |
| Capping par percentiles | 1 <sup>er</sup> et 99 <sup>e</sup> percentile (ou 5e/95e) |
| Capping par IQR         | $Q1 - 1.5 \times IQR$ et $Q3 + 1.5 \times IQR$            |

```
import pandas as pd

# Exemple : série de revenus avec valeurs extrêmes
revenus = pd.Series([3000, 3200, 3100, 3050, 70000])

# Définir les seuils de capping (1er et 99e percentiles)
lower_bound = revenus.quantile(0.01)
upper_bound = revenus.quantile(0.99)

# Appliquer le capping
revenus_capped = revenus.clip(lower=lower_bound, upper=upper_bound)

print("Avant capping :", revenus.tolist())
print("Après capping :", revenus_capped.tolist())
```

Avant capping : [3000, 3200, 3100, 3050, 70000]  
Après capping : [3002, 3200, 3100, 3050, 67328]

# Gestion des valeurs aberrantes (Outliers)

## 3.2. Limiter par Capping (Winsorization)

La **transformation** consiste à **appliquer une fonction mathématique** à une variable pour **réduire l'asymétrie** de sa distribution et **atténuer l'effet des outliers**.

**Pourquoi?**

- Pour **rendre la distribution plus normale**
- Pour **stabiliser la variance**
- Pour **améliorer la performance** des modèles linéaires ou paramétriques
- Pour **atténuer l'influence des grandes valeurs** sans les supprimer

**Quand l'utiliser ?**

- Sur des variables **strictement positives** (ex. : revenus, nombre de clics, population)
- Quand la variable présente une **forte asymétrie positive (skewness > 1)** avec longue queue à droite
- Quand la variance est **non constante** (hétéroscédasticité)

**Quelle transformation:** Logarithmique, Racine carrée, Puissance inverse, Box-Cox, Yeo-Johnson

## Gestion des valeurs aberrantes (Outliers)

### 3. Le dilemme : Supprimer ou Limiter :

| Stratégie                            | Quand l'utiliser                                              | Exemples pratiques                               |
|--------------------------------------|---------------------------------------------------------------|--------------------------------------------------|
| <b>Suppression</b>                   | Si l'outlier est clairement une erreur                        | Valeur de température de 800 °C chez un humain   |
| <b>Capping (Winsorization)</b>       | Si on veut <b>limiter l'impact sans supprimer</b>             | Ramener tous les revenus > 99e percentile au 99e |
| <b>Transformation (log, sqrt...)</b> | Si les outliers sont légitimes mais déforment la distribution | Revenu, population, ventes                       |
| <b>Imputation</b>                    | Si la valeur est aberrante car manquante/mal enregistrée      | Remplacer 999 par la médiane du groupe           |
| <b>Conservation + Encodage</b>       | Si l'outlier peut avoir une valeur informative                | Ajouter une colonne is_outlier = 1/0             |
| <b>Modèles robustes</b>              | Si on ne veut pas nettoyer mais modéliser avec robustesse     | Régression robuste, arbre de décision            |

# Gestion des valeurs aberrantes (Outliers)

## Bonnes pratiques :

- Toujours **visualiser** les données avant de supprimer ou transformer.
- **Documenter** les décisions : ne jamais supprimer un outlier sans justification.
- Si possible, **demandeur une confirmation métier** : certaines valeurs extrêmes sont **valides** (ex. : salaires de PDG, ventes exceptionnelles).
- En apprentissage automatique, **tester les performances** du modèle avec et sans outliers.

# Laboratoire formatif



## Laboratoire formatif

- Charger le fichier CSV dans un DataFrame.
- Afficher le nombre de valeurs manquantes par colonne.
- Imputer les valeurs manquantes, si besoin :
  - Imputer Age par la moyenne.
  - Imputer Embarked .
- Détecter les valeurs aberrantes dans la colonne Fare avec la méthode IQR.

Il faut:

- Identifier les données manquantes et leur type (MCAR, MAR, MNAR)
- Imputer les valeurs manquantes avec des méthodes adaptées
- Détecter les valeurs aberrantes par des outils visuels et statistiques
- Appliquer des méthodes de traitement (suppression, capping, transformation)